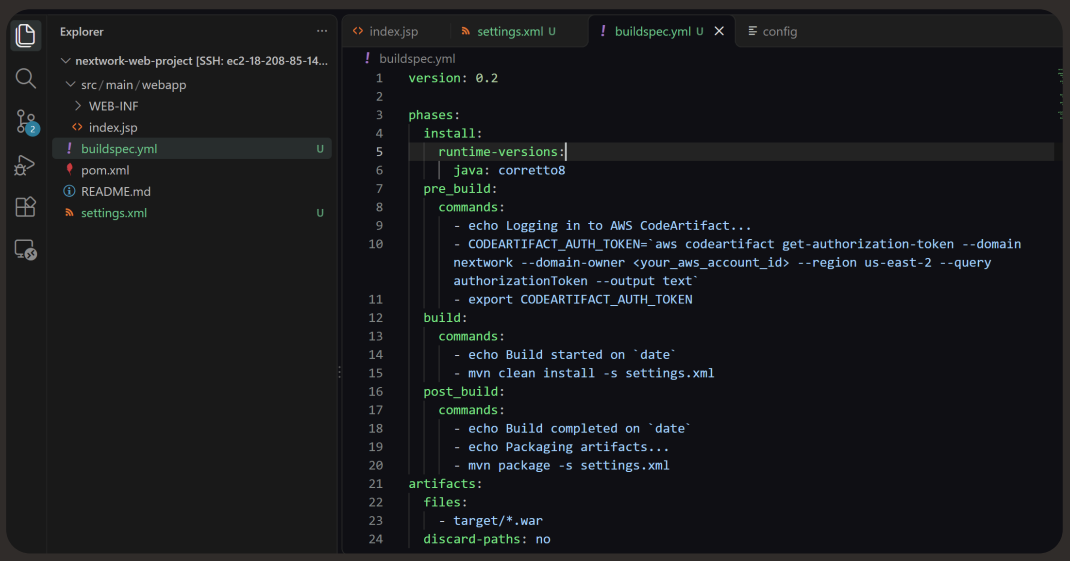
# CONTINUOUS INTEGRATION WITH CODEBUILD

By Maryna Maksimuk | October 2026



## Introducing Today's Project!

Today, I will create and configure an AWS CodeBuild project and connect it to my GitHub repository. I will create a buildspec.yml file to define the build and testing process, allowing CodeBuild to automatically build and test my Java application whenever the process runs.

## Key tools and concepts

I learned how to use AWS CodeBuild, CodeArtifact, Amazon S3, CloudWatch Logs, IAM roles and policies, and AWS CodeConnections to GitHub. I also gained hands-on experience configuring a CI build process, managing Maven dependencies, creating a buildspec.yml file, automating tests with a Bash script, troubleshooting permission errors, and verifying successful builds.

## Project reflection

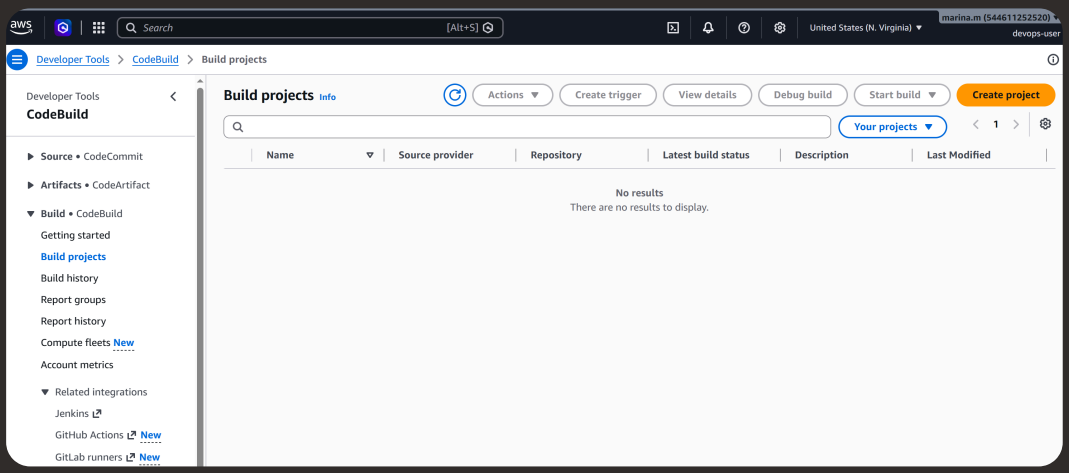
It took me approximately 3 hours to complete this project, including configuring AWS CodeBuild and CodeArtifact, connecting GitHub, troubleshooting IAM permission issues, and automating tests for my Java web application.

I plan to start my next project on Monday, after the weekend, and continue building my AWS and DevOps skills through hands-on practice.

# Setting up a CodeBuild Project

A CI service, such as AWS CodeBuild, automatically builds and tests code whenever developers make changes, helping detect errors early and improve software quality.

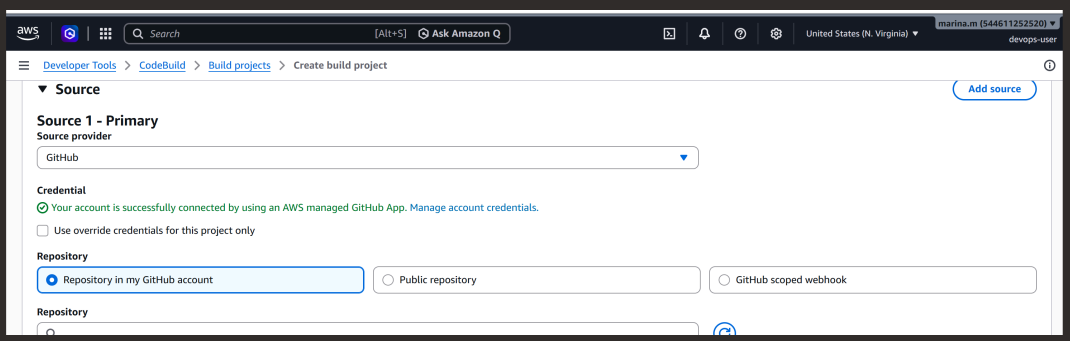
The source of my CodeBuild project is my GitHub repository, where my Java web application's code is stored and retrieved from when CodeBuild runs a build.



# Connecting CodeBuild with GitHub

A GitHub App is recommended because it provides secure, fine-grained access to repositories, avoids managing personal access tokens, and makes it easier for AWS to connect to GitHub automatically.

AWS CodeConnections securely linked my AWS account to GitHub, allowing CodeBuild to access my repository and retrieve my application's source code without manually managing credentials.



# CodeBuild Configurations

## Environment

I built my AWS CodeBuild project by connecting it to my GitHub repository, selecting Amazon Linux as the operating system, choosing the Standard runtime with the Corretto 8 image to support my Java application, and creating a new service role to give CodeBuild the permissions it needs to run the build process.

## Artifacts

The S3 bucket will store the build artifact, typically the packaged Java web application file (.war file), which can be used later in the deployment process.

## Packaging

We package build artifacts as a ZIP file to bundle all the necessary application files into one compressed file, making it easier to store in Amazon S3 and transfer to later stages of the CI/CD pipeline.

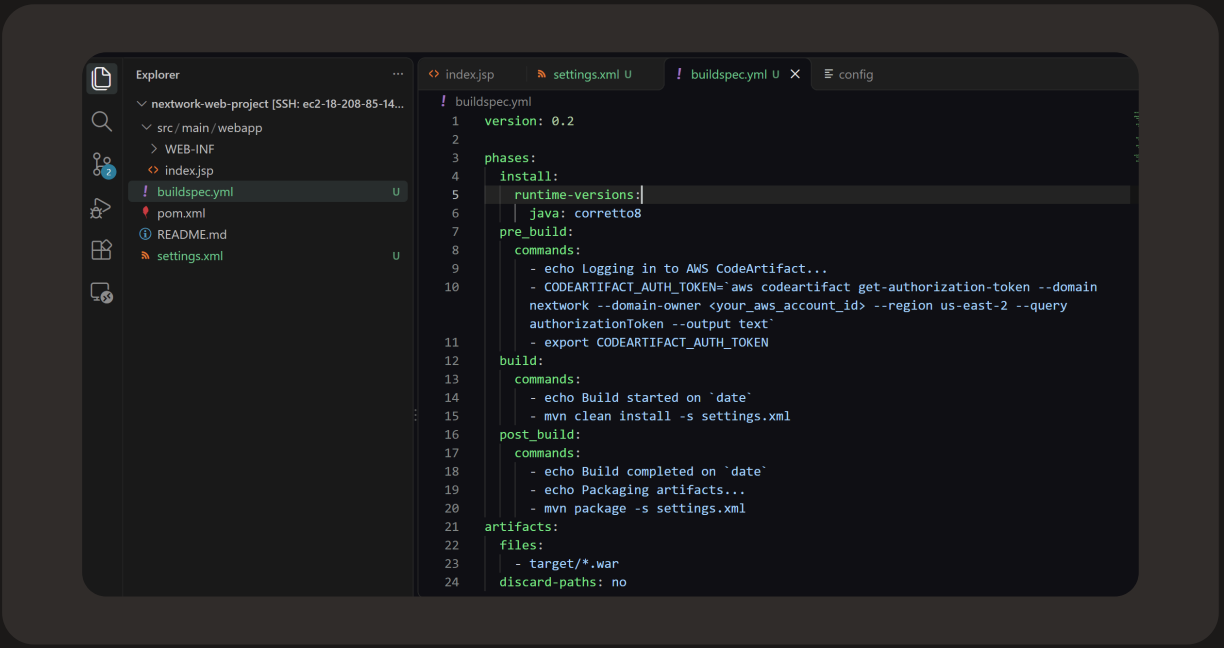
## Monitoring

We use Amazon CloudWatch Logs to monitor CodeBuild's build process, view build output, identify errors, and troubleshoot problems when a build fails.

# buildspec.yml

My first build failed because I didn't have a buildspec.yml file in the root of my repository, so CodeBuild didn't know which commands to run to build and test my Java application.

The different phases in buildspec.yml define the steps CodeBuild follows: install sets up dependencies, pre_build prepares the build environment, build compiles and tests the application, and post_build performs final tasks such as packaging the build artifacts.

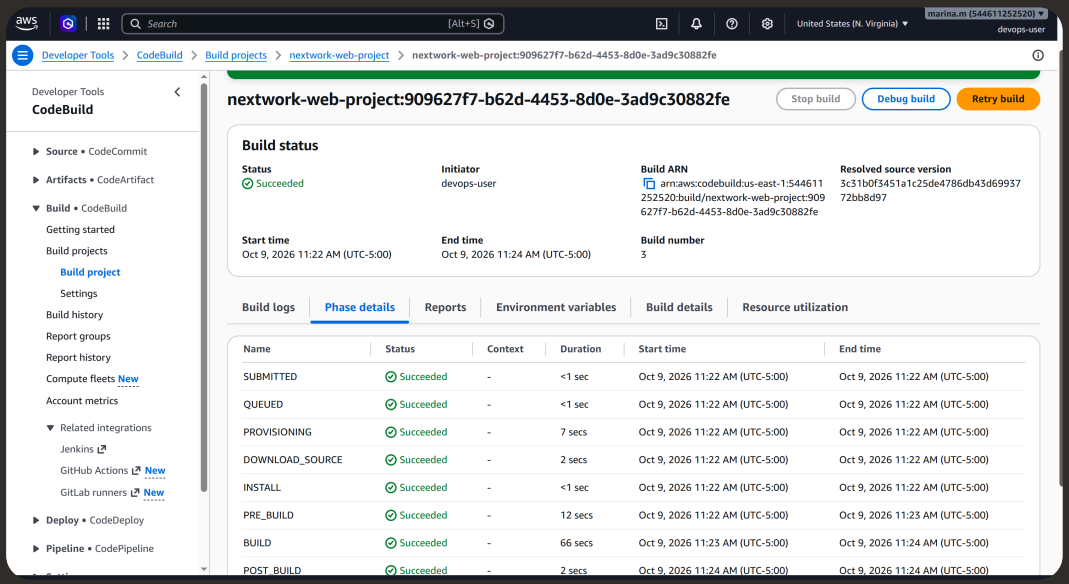


# Success!

The build failed because CodeBuild's IAM service role didn't have the necessary permissions to access AWS CodeArtifact and download the Maven dependencies required to build my Java application.

I resolved the error by adding the necessary AWS CodeArtifact permissions to the CodeBuild service role through an IAM policy, allowing CodeBuild to access the repository, download dependencies, and build my Java application successfully.

I checked the Amazon S3 bucket to verify that the build artifact was uploaded successfully, and the build created a ZIP file containing my Java web application's packaged files.



# Automating Testing

The script runs three simple tests to verify that my Java web application has the correct project structure. It checks that the src directory and index.jsp file exist, then runs a basic validation test that always passes. If a required file or directory is missing, the script reports a failure and stops; if all checks pass, it confirms that all tests passed.

I added a run-tests.sh file to the root of my repository with simple validation tests, then updated buildspec.yml to execute the script during the CodeBuild process so the tests run automatically with each build.

I verified that testing was automated by running the CodeBuild project and checking the build logs to confirm that run-tests.sh executed automatically and all tests passed successfully.

